scitech-librarian — User Manual

version 3.5.0

2026-09-04

Contents

English · Português (Brasil) · Español · Deutsch · Français

1. What it is

scitech-librarian is a reproducible literature-search instrument for science and engineering. You write a structured query once; it runs the query against up to nine bibliographic databases through their documented APIs, archives everything (records, the exact query string sent to each database, hit counts, a log), and writes a literature-search report with a PRISMA 2020 flow diagram. Over months, the runs, plus records you obtained by other means, accumulate in a **research directory** that the same report can describe as a whole — what each search added, what each database contributed, how counts drifted, which venues matter.

It is five Python scripts plus two shared modules (`render.py`, `i18n.py`), with no dependencies beyond the standard library. There is nothing to install: copy the files, supply your keys, write `queries.json`, run.

File	Role
`librarian.py`	run a search; archive a run; call the report
`project.py`	research directory: index, ingest external records, status
`report.py`	reports for one run or the whole directory; PRISMA; filters
`journals.py`	journal metrics (impact-factor-like figures) per year
`wos_manual.py`	Web of Science by hand (no usable free API)
`render.py`	Markdown / HTML / LaTeX / text renderers and the PDF chain (imported by `report.py`)
`i18n.py`	report languages: the en / pt-BR / es / de / fr catalogue (imported by `report.py`; §7.8)

For AI agents. `AGENTS.md` at the repository root is the complete machine-oriented description of the tool. If you work with a coding agent (Claude Code, Codex, Cursor…), tell it: *"Read AGENTS.md, then run a novelty check on X"* — it contains the commands, file schemas, workflows and the rules the agent must not break.

2. Installation and setup

Requirements: Python 3.9 or newer. Optional, for typeset PDF reports: a LaTeX distribution (xelatex, lualatex or pdflatex) or pandoc; without them the PDF is produced by a built-in plain-text writer.

```
git clone https://github.com/fabiocampolim-design/scitech-librarian
cd scitech-librarian
cp .env.example .env            # fill in what you have (or use the environment)
cp queries.example.json queries.json
python librarian.py --selftest
```

Keys are read from the process environment. Copy `.env.example` to `.env` and fill that in, or set the same variable names in your shell, in your agent's or launcher's configuration, or as CI secrets — `.env` only supplies what the environment has not already set, so a variable set outside always wins and the file is optional.

Key	Needed for	How to get it
`CONTACT_EMAIL`	polite-pool access to OpenAlex/Crossref/Unpaywall	your address
`ADS_TOKEN`	NASA ADS	free, https://ui.adsabs.harvard.edu/user/settings/token
`SCOPUS_API_KEY`	Scopus (+ institutional network/VPN)	free, https://dev.elsevier.com/apikey/manage
`SCOPUS_INSTTOKEN`	Scopus without VPN	ask your library
`S2_API_KEY`	faster Semantic Scholar	optional

Key	Needed for	How to get it
`CORE_API_KEY`	CORE — optional; anonymous callers work but are rate-limited	free, https://core.ac.uk/services/api
`WOS_STARTER_KEY`	Web of Science Starter API (restricted grammar)	rarely worth it

Six backends (OpenAlex, arXiv, INSPIRE-HEP, Semantic Scholar, Crossref, CORE) need no key and no institution. `python librarian.py --list` reports which keys were found by either route; `--selftest` proves they work.

Drop-in use inside another project. Put the seven files in a `tools/` subdirectory; `.env`, `queries.json` and `lit/` are then looked for in the parent directory.

3. Concepts

Block. One structured query: a list of synonym groups combined with AND, each group a list of synonyms combined with OR. A block has a name (`A`, `CD`, `NOV`…), a title and a note. Blocks live in `queries.json`.

Run. One execution of `librarian.py`: every selected block against every selected backend, archived under `lit/runs/<timestamp>/`.

Research directory. A folder (default `lit/`, choose another with `--outdir`) holding all the runs of one project, the records ingested from outside, the project index (`project.json`), the PRISMA screening numbers (`screening.json`), journal metrics, reports and audit logs. One directory per project; a lab has several.

Manual source. Records that did not come from a run: a Zotero or Mendeley export, a colleague's RIS file, a Web of Science session, a reference list. Ingested with `project.py ingest`, they keep their provenance (who, when, where from, method) and appear in every report as one more source, and in the PRISMA flow in the right column.

Record. The common schema every file uses: `title year doi journal authors url abstract cited_by issn block backend`. Merged project records also carry `found_by` (which sources found it) and `first_seen`.

Level. How much a report contains: `simple` (a few pages), `intermediate` (every unique record plus analyses), `full` (everything, abstracts included — hundreds of pages for large projects).

4. Writing queries

`queries.json`:

```
{
  "PSC": {
    "title": "Perovskite solar cell degradation under humidity",
    "note": "grounding block -- expect thousands",
    "groups": [["perovskite solar cell", "halide perovskite photovoltaic"],
               ["degradation", "stability"],
               ["humidity", "moisture"]]
  },
  "NOV": {
    "title": "novelty check: origami metamaterials as topological acoustic pumps",
    "note": "a SMALL number is the good outcome; read every hit",
    "groups": [["origami", "kirigami"], ["acoustic metamaterial", "phononic crystal"],
               ["topological pumping", "edge state"]],
    "arxiv_groups": [0, 2]
  }
}
```

Rules of thumb:

- Do not quote terms; the tool quotes for each database's grammar.
- A lone generic word (`model`, `structure`, `system`) in its own group is the usual cause of counts in the tens of thousands.
- `arxiv_groups` names which (at most two) groups arXiv receives; arXiv hangs on deeply nested booleans. Default: the first two. arXiv is paged 100 records at a time with a 3 s pause, so a large `--limit` is slow there.
- The most informative block is a deliberate intersection of two literatures you suspect do not talk to each other. A near-zero result is a finding — *if* you then read every hit.
- Proximity operators (`NEAR/n`, `W/n`) are not expressible; if your paper needs them, keep hand-written Web of Science / Scopus strings alongside and quote those.

5. Running a search

```
python librarian.py                        # all blocks, every configured backend
python librarian.py --counts-only          # hit counts only (seconds)
python librarian.py --blocks NOV CD        # selected blocks
python librarian.py --backends openalex ads
python librarian.py --skip arxiv           # drop a misbehaving backend
python librarian.py --limit 500            # records per block and backend (default 300)
python librarian.py --pdfs --pdf-blocks NOV  # legal OA-PDF links via Unpaywall
python librarian.py --keep-junk            # keep non-curated venues (Zenodo, SSRN…)
python librarian.py --outdir lit_topomat   # another research directory
python librarian.py --report-level intermediate --report-format md html pdf
python librarian.py --report-lang pt-BR    # report in Portuguese (en, pt-BR, es, de, fr; §7.8)
python librarian.py --no-report
python librarian.py --queries other.json      # another query file (default ./queries.json)
python librarian.py --backends-file b.json    # another backends config; --init-backends writes the defau
python librarian.py --timeout 60              # per-request timeout in seconds (default 45)
python librarian.py --list                    # blocks and backend readiness, then exit
python librarian.py --selftest                # ping every backend, then exit
```

Complete parameter list: `python librarian.py --help`. Every option has a default; `--outdir`, `--verbose`, `--quiet` and `--log-dir` exist on every script.

What a run writes (`lit/runs/<stamp>/`):

File	Content
`counts.json`, `counts.md`	hit counts per block and backend; paste-ready table
`queries.json`	the exact query string sent to each backend
`blocks.json`	the block definitions used
`meta.json`	settings, backends and endpoints, version, timing
`records/<block>_<backend>.json`	raw records per backend (after the venue filter)
`ris/<block>_<backend>.ris`	per-block RIS for Zotero/Mendeley/EndNote
`all_records.json/.csv/.ris`	deduplicated, sorted by citations
`all_records.bib`, `all_records.csl.json`	the same set as BibTeX and CSL-JSON
`junk.json`	records removed by the venue filter, with their venues
`prisma.json`	template for the manual PRISMA stages
`run.log`	everything printed
`report.*`	the report (see §7)

Plus `lit/counts_history.csv` (one row per block/backend/run, for drift) and `lit/logs/librarian_<stamp>_<pid>.log` (audit log: invocation, versions, every message).

Counts are checkpointed after every API call and Ctrl-C is safe: a hang late in a long run loses nothing.

6. The research directory

6.1 Index

```
python project.py init --name "Topological materials review" --description "…"
python project.py status
```

`status` lists every member (runs and manual sources) with date, record count, method and label, the inbox state, and the last report. Members are discovered by listing the directory — nothing has to be declared. `project.json` holds only what cannot be discovered:

```
{"name": "…", "description": "…", "created": "2026-08-28",
 "exclude": ["20260814T223331"],
 "labels": {"20260828T095041": "August full scan"},
 "block_aliases": {"X": "CD"},
 "defaults": {"level": "simple", "format": ["md"], "metric": "openalex_2yr"}}
```

```
python project.py exclude 20260814T223331      # a test run you do not want in reports
python project.py label 20260828T095041 "August full scan"
python project.py alias X CD                    # block renamed between runs
python project.py oa                            # Unpaywall pass over every member that lacks OA data
python project.py oa --members 20260828T095041  # restrict the pass to these member ids
```

`oa` is the post-hoc open-access lookup: runs made without `--pdfs` and manual sources get `is_oa` / `oa_pdf` fields (legal copies only, cached in `unpaywall_cache.json`), which the report's OA statistics and `--oa-only` then cover for the whole project.

6.2 Bringing records in from outside

Three ways, all ending in `lit/manual/<name>/` with the original file, a `records.json` in the common schema and a `source.json` with provenance:

1. **Command line** — the fully described way:

   ```
   python project.py ingest export.ris --name zotero-aug --block CD \
          --method citation --who "A. Colleague" --origin "Zotero group library" \
          --note "reference lists of the three key papers"
   ```

   Several files may be given; `--kind` overrides extension detection (`ris`, `bibtex`, `csv`, `json`).

2. **Inbox** — drop files into `lit/inbox/` and run `python project.py ingest --inbox`; each file becomes a source named after it (add `--method` etc. to apply to all).

3. **Web of Science** — `python wos_manual.py ingest` reads the RIS files you exported from the WoS interface and registers them as manual sources with `method=database`.

Accepted formats: RIS (Zotero, Mendeley, EndNote, Web of Science, Scopus), BibTeX, CSV with a header row (Scopus and WoS column names recognised; else `title, year, doi, journal, authors, url, abstract, block, cited_by`), and JSON record lists (for instance `all_records.json` from a colleague's run).

`--method` follows PRISMA 2020's categories for records identified via other methods: `database` (a database export — joins the databases column), `citation` (reference lists, citing papers), `website`, `organisation`, `expert` (a colleague's recommendation), `other`.

You may also hand extra files to a single report without storing them: `report.py --records file.ris`.

6.3 From Zotero, Mendeley and EndNote

Out: every run writes RIS (`all_records.ris`, per-block `ris/`), BibTeX (`all_records.bib`) and CSL-JSON (`all_records.csl.json`); import with File → Import. Abstracts, DOIs and URLs are carried, and the block name arrives as a keyword (`block:NOV`) so the imported items are pre-tagged.

In: export a collection as RIS (Zotero: right-click → Export Collection → RIS; Mendeley: File → Export → RIS; EndNoteX: File → Export → RefMan RIS) and ingest it as above. There is no live connection to the Zotero API (roadmap).

7. Reports

7.1 One run

```
python report.py lit/runs/20260828T095041
python report.py --latest --level full --format html pdf
```

7.2 The whole research directory

```
python report.py --project
python report.py --project --outdir lit_topomat --level intermediate --format md html
```

Reports go to `lit/reports/<stamp>-<level>/`. The project report adds a **Sources** table (every run and manual source, its date, method, records and "new here" — the unique records no earlier source had found), a **Timeline** (per-block counts over runs; when records entered the project), a PRISMA flow with both identification columns, and, when `journals.py` has been run, journal metrics.

7.3 Levels

Level	Sections
`simple`	metadata; sources; search strategy with the exact string per backend; results summary; timeline; PRISMA 2020 flow + PRISMA-S checklist; top 10 records per block; suggestions
`intermediate`	+ every unique record; source overlap ("found only here"); year / venue / author distributions; journal metrics; filtered venues; errors; open-access stats; count history
`full`	+ every record with full abstract, author list and which sources found it; per-source raw lists before deduplication; the filtered records; backend configuration; project.json and source.json files; the run log; environment

Sizes, from the shipped sample (four blocks, three CC0 databases, 1,226 unique records): 6, 68 and 427 PDF pages.

7.4 Formats

`md` (Markdown; renders on GitHub), `html` (self-contained, light/dark, printable, SVG diagram), `tex` (LaTeX with a TikZ diagram), `pdf`, `txt` (plain text, ASCII diagram). The PDF is compiled from the LaTeX with xelatex, lualatex or pdflatex if one is installed, else with pandoc, else by a built-in writer that lays out the text version — the option never fails.

7.5 Filters

Option	Effect
`--since DATE`, `--until DATE`	keep members (runs / manual sources) searched in the window

Option	Effect
`--latest`	most recent member only (project); newest run (single mode)
`--diff`	keep only records *first seen* inside the window — "what the searches since DATE added"
`--year-from Y`, `--year-to Y`	publication year
`--backends a b`	databases / sources to include (manual sources are `manual:<name>`)
`--blocks A CD`	blocks to include
`--sources auto|manual|all`	member kinds
`--records FILE…`	extra RIS/BibTeX/CSV/JSON as a transient manual source
`--metric NAME --min-metric X`	keep records whose venue metric is at least X (see §8)
`--min-citations N`	citation threshold
`--oa-only`	only records with a legal open-access copy (needs `--pdfs` or `project.py oa` data)
`--top N`, `--sort cited|year|metric`	table size and order
`--basename`, `--out`	file stem and output directory

Filters are listed in the report's metadata table and in PRISMA-S item 9, so a filtered report is never mistaken for the whole search.

7.6 PRISMA

The report carries a PRISMA 2020 flow diagram (SVG in HTML, TikZ in LaTeX/PDF, ASCII in Markdown and text) and a PRISMA-S search-reporting checklist. The tool fills what it can know: records identified per database (summed over runs in project mode), records identified via other methods (manual sources by method), records retrieved, removed by automation (the venue filter), duplicates removed, records left to screen. It is explicit that *identified* (what each database reports) and *retrieved* (what was downloaded within `--limit`) differ.

The stages only a human can know are read from `prisma.json` (single run) or `screening.json` (research directory); a template with `null` values is written on the first report. Fill in the integers as screening progresses and re-run the report:

```
{"records_screened": 2216, "records_excluded": 2100,
 "reports_sought": 116, "reports_not_retrieved": 4, "reports_assessed": 112,
 "excluded_reasons": {"not topological": 60, "no experiment": 30},
 "other_sought": 12, "other_not_retrieved": 0, "other_assessed": 12,
 "other_excluded_reasons": {"duplicate of included": 3},
 "studies_included": 22, "reports_included": 31,
 "citation_searching": "reference lists of the 22 included studies",
 "prior_work": "none", "peer_review": "search strategy reviewed by the librarian"}
```

7.7 Suggestions

Rule-based, at the end of every report: failed backend calls, blocks with thousands of hits, novelty-sized blocks (read every hit), `--limit` cap hits, a database with a high share of filtered venues, no citation-grade backend, open-access lookup not run, PRISMA stages unfilled, no journal metrics, count drift between runs, and — in project mode — the absence of any manual source.

7.8 Languages

```
python report.py --latest --lang pt-BR
python report.py --project --lang de --format pdf
python librarian.py --report-lang fr            # the report written at the end of a run
```

`--lang` (`report.py`) and `--report-lang` (`librarian.py`) take `en` (default), `pt-BR`, `es`, `de` or `fr`; a research directory can fix its own default with `"defaults": {"lang": "es"}` in `project.json`, and an explicit flag wins over it. Only the report's own wording changes — headings, table headers, the PRISMA 2020 stages and the flow diagram in every format, the PRISMA-S checklist, the explanatory paragraphs and the suggestions — together with the language's thousands separator. What the tool found or was given is reproduced exactly as it is, whatever the language: record titles, abstracts, authors and venues, your block names and notes, the exact query strings, backend names, the flags quoted in the text, file names, the JSON dumps and the embedded run log. Console output, `run.log` and the audit logs are always English, so runs made in different languages stay searchable together.

8. Journal metrics

```
python journals.py fetch                              # every journal seen in the directory
python journals.py fetch --providers openalex --refresh
python journals.py import-scimago scimagojr_2024.csv --year 2024 [--all]
python journals.py import-jcr JCR_JournalResults_*.csv       # Journal Citation Reports downloads
python journals.py import-csv other.csv --provider my_metric --year 2023 --name-col Journal --
value-col Value        [--issn-col ISSN] [--delimiter ";"]                  # any name/value table; ISSN
python journals.py list --missing jcr_if                       # journals still to look up by hand
python journals.py show --metric scopus_citescore
```

Store: `lit/journals/metrics.json`, one entry per journal keyed by ISSN (else normalised name), values kept **per year and never overwritten** — refetch next year and the report shows the series.

Provider	Key	Metrics	History
`openalex`	none	`openalex_2yr` (2-year mean citedness, an impact-factor-like figure), `openalex_h`, works/citations by year	snapshot under the fetch year
`scopus`	`SCOPUS_API_KEY`	`scopus_citescore`, `sjr`, `snip`	full history per year
`scimago`	none; download the year's CSV from scimagojr.com	`sjr`, `scimago_h`, quartile	one file per year
`jcr`	licence	`jcr_if`	import only

The Journal Impact Factor (Clarivate JCR) is proprietary: there is no free API and the tool will not scrape it. Licensed users download CSVs from the JCR *Browse journals* page (600 rows per download; slice by category, then by quartile) and import them with `journals.py import-jcr FILE...` — columns and the JIF year are detected. `journals.py list --missing jcr_if` prints the journals in your directory still without a value, which is the list to look up. The full protocol is in `docs/JCR_IMPORT.md`. For a metric that covers every journal, the SCImago CSV (~30,000 journals, one download) is the practical route; `--all` imports the whole file, the default imports only journals seen in your records.

In reports: a metric column in record tables, "venues in this set by metric", an evolution table for venues with two or more years on file, and the `--min-metric` filter. `--metric` selects which (default `openalex_2yr`, or `defaults.metric` in `project.json`).

9. Web of Science

The full `TS=`/`NEAR` grammar is in the Expanded API, rarely licensed; the free Starter tier rejects complex booleans. Web of Science is therefore a manual job, made small:

```
python wos_manual.py prep      # query files + CHECKLIST.md in WoS grammar
```

```
python wos_manual.py walk      # copies each query to the clipboard in turn
python wos_manual.py ingest    # RIS exports -> records, registered as manual sources
python wos_manual.py status
python wos_manual.py prep --queries other.json   # a different query file (default ./queries.json)
```

The checklist encodes the UI settings that silently break queries (Core Collection, Advanced search, editions, tagged vs bare form).

10. Logs and audit

Every script writes `<outdir>/logs/<script>_<stamp>_<pid>.log` with the exact invocation, tool and Python versions, the research directory, every warning and error, and the outcome. Console output is small by default; `--verbose` shows everything, `--quiet` only warnings and errors; `--log-dir` moves the logs. Runs additionally keep `run.log` (the console transcript) in the run directory.

11. Workflows

A novelty check (an afternoon). Write 1–3 cross-query blocks; `--counts-only`; tighten anything in the thousands; full run with `--pdfs`; read the Suggestions; read every hit of the small blocks by hand; record what you screened in `prisma.json`; re-run `report.py`; import the RIS into Zotero.

A systematic search over a project (months). `project.py init`. Rerun `librarian.py` at intervals with the same `queries.json`. Ingest the Web of Science sessions and colleagues' exports. `report.py --project` for the picture; `--project --since <last report> --diff` for what is new; `journals.py fetch` yearly. Fill `screening.json` as you go; the PRISMA diagram completes itself, ready for the supplement.

A lab. One research directory per project (`--outdir`); each has its own index, screening and reports. Inbox folders let collaborators drop exports without learning the tool. There is deliberately no cross-project merge: different questions, different blocks.

A worked example. `docs/WALKTHROUGH.md` runs a real project from `queries.json` to a completed PRISMA diagram, every command included.

With an AI agent. Point it at `AGENTS.md`; ask it to draft `queries.json` from your research question, run the scans, and walk the report with you. The structured query file, the JSON archives and the report were designed to be written and audited by an agent.

12. Features and limitations

Features: one structural query rendered into nine native grammars; databases as JSON configuration (`--init-backends`); archived, citable runs with exact query strings and count history; checkpointing and safe Ctrl-C; a venue filter with receipts; five keyless backends; NASA ADS and INSPIRE for physics; legal OA-PDF links via Unpaywall; three-level reports in five formats with PRISMA 2020 and PRISMA-S; research directories with manual sources, provenance, timeline and differential reports; journal metrics with a per-year series; audit logs; an offline test suite (313 checks) and CI.

Limitations, all by design or by the world:

- Counts are not comparable across databases; proximity operators are dropped. Discover here; quote one database in the paper.
- Scopus results require institutional entitlement (network/VPN). Web of Science's API is rarely licensed; use the manual path.
- arXiv receives at most two groups per block.
- `--limit` caps records per block and backend (most-cited first); large blocks are a slice, not the complete set. Raise it when you need completeness.
- OpenAlex indexes non-curated repositories (~15 % of its records); filtered by default, kept in `junk.json`.

- No download of PDFs (Unpaywall links only), no snowballing, no citation graph, no live Zotero/Mendeley connection (roadmap); BibTeX and CSL-JSON are written, not read back from a Zotero library.
- Journal metrics: OpenAlex values are snapshots; the JCR Impact Factor is proprietary and import-only; matching journals by name is imperfect when a record has no ISSN.
- Deduplication is by DOI, else the first 90 characters of the title; preprint/published pairs with different titles survive as two records.
- Google Scholar is not and will not be a backend (no API; scraping violates its terms).

13. Testing

```
python tests/test_librarian.py
```

Offline, stdlib only, no keys: backends run against canned API responses, the report generator against synthetic run and research directories, and every script's command line is exercised end to end. The file is also a pytest module (`pytest tests/`). CI runs pyflakes and the suite on Linux, Windows and macOS under Python 3.9 and 3.13.

14. Licence and conduct

Apache License 2.0. The tool is built to make respecting each database's terms of service the easy path: documented APIs only, rate limits honoured, a contact address in every request, no scraping, no paywall circumvention.